

TECHNICAL DOCUMENTATION

Full Deployment and Implementation Guide

PostgreSQL/PostGIS operations, Git workflow, two-project Vercel deployment, migration strategy, security, testing, and the implementation plan.

Current database status	Not reachable on this machine. PostgreSQL port 5432 refused the connection, and the local Docker daemon requires administrator access. The schema and deployment configuration are ready, but rows could not be inserted during this check.
-------------------------	---

Repository: /home/amisiespoir-07/Dev/singapore
Branch: main | Git remote: not configured

1. Executive status

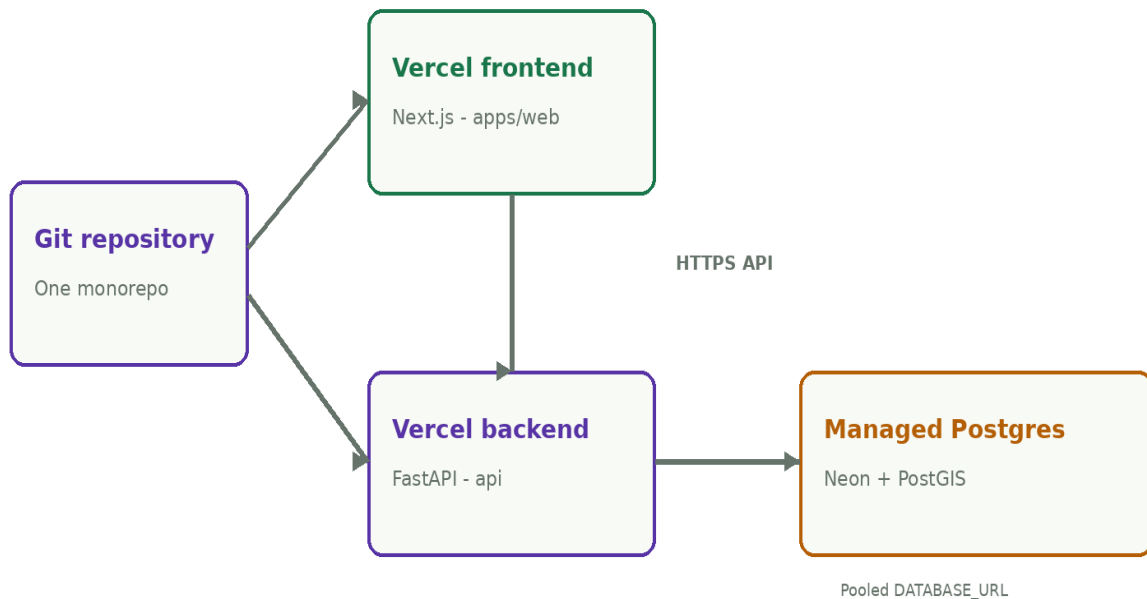
Area	Status	Evidence or action
PostgreSQL connection	BLOCKED	No service is listening on local port 5432.
Docker service	BLOCKED	Daemon access requires an administrator password.
Schema	READY	Alembic emits valid PostgreSQL/PostGIS DDL.
Spatial model	READY	Geography points and GiST indexes are defined.
Application build	READY	Python compilation and Next.js production build pass.
Database tests	READY TO RUN	Four integration tests are collected; they need a live PostGIS database.

Truthful interpretation

The database integration is implemented but not proven against a live database on this host. Do not call the deployment production-ready until migrations, seed, spatial matching, and grouped-order tests pass on a running PostGIS instance.

2. Target architecture

Production deployment architecture



- One Git repository drives two Vercel projects.
- Frontend project root: apps/web. It builds Next.js and calls NEXT_PUBLIC_API_URL.
- Backend project root: api. Vercel discovers index.py and runs FastAPI as a Python Function.
- The backend connects to managed PostgreSQL with PostGIS through a pooled DATABASE_URL.
- Alembic migrations run as an explicit release step, never during every function startup.
- Production, preview, and development use separate credentials and preferably separate database branches.

3. Persistent domain model

Table	Purpose	Critical constraints
users	Buyer, farmer, admin identities	Unique email; role check; active flag
auth_tokens	Persistent bearer sessions	Token primary key; delete with user
farmer_profiles	Public farmer details	One profile per farmer
farms	Farm identity and geographic point	Geography POINT(4326); GiST index
lots	Available produce inventory	Quantity ≥ 0 ; price > 0 ; active flag
buyer_requests	Demand, radius, pickup point	Quantity/radius > 0 ; status state machine
match_plans	Persisted matching decision	One current plan per request
allocations	Per-farmer quantity and score	References request, lot, farmer
orders	Reserved inventory and fulfilment state	Quantity > 0 ; status check; group ID

Geospatial query

```
ST_DWithin(farms.location, buyer_request.location, radius_km * 1000)
ST_Distance(farms.location, buyer_request.location)
```

PostGIS geography calculations use metres. Coordinates are written as POINT(longitude latitude), not latitude first.

4. Dummy dataset and expected behavior

The controlled seed command creates four users, two farms, four lots, two sample orders, three tokens, and one buyer request.

Object	Identifier	Key values
Buyer	usr_demo_buyer	buyer@example.com
Farmer 1	usr_demo_farmer_1	Green Valley Farm
Farmer 2	usr_demo_farmer_2	Sunrise Fields
Admin	usr_demo_admin	admin@example.com
Buyer request	req_demo_tomatoes	100 kg tomatoes; max 0.80; 50 km
Sample order	ord_demo_1	20 kg; ACCEPTED
Pending order	ord_demo_pending	10 kg; PENDING

Allocation	Quantity	Distance	Price	Approx. score
Green Valley tomatoes	60 kg	0.90 km	0.70/kg	0.352
Sunrise tomatoes	40 kg	1.39 km	0.75/kg	0.395

Atomic confirmation

The finalization route locks the buyer request and eligible lots, recalculates the plan, reserves quantities, creates linked orders, and updates the request in one transaction.

5. Local database operations

Initial setup

```
sudo docker compose up -d database
python -m venv .venv
.venv/bin/pip install -r api/requirements.txt
.venv/bin/alembic upgrade head
.venv/bin/python -m api.scripts.seed_demo --yes-reset
```

Verification

```
.venv/bin/pytest api/tests
.venv/bin/uvicorn app.main:app --app-dir api --reload
curl http://127.0.0.1:8000/health
```

Migration rules

- Generate a new migration for every schema change.
- Review generated SQL before applying it.
- Test upgrade from an empty database and from a copy of the previous schema.
- Do not automatically downgrade production after writes have used a new schema.
- Back up before destructive changes and use expand-migrate-contract for incompatible releases.

6. Push the repository

The repository currently has no Git remote. Create an empty GitHub, GitLab, or Bitbucket repository first.

```
git status
git add .
git commit -m "Add PostgreSQL PostGIS persistence and deployment guides"
git remote add origin https://github.com/YOUR_ACCOUNT/farmer-marketplace.git
git push -u origin main
```

Before pushing

- Confirm .env and .venv are ignored.
- Confirm DATABASE_URL and tokens do not appear in git diff or history.
- Run the backend tests against PostGIS and npm run build in apps/web.
- Review the large untracked directories and commit only intended project assets.

Current repository condition

Most project files are untracked. The first commit should be reviewed carefully because git add . will include every non-ignored file.

7. Provision managed Postgres with PostGIS

- In Vercel Marketplace, install a managed Postgres integration such as Neon.
- Create a production database and a separate preview/development branch.
- Use the provider's pooled connection string for DATABASE_URL in the backend function.
- Confirm PostGIS is available, then run CREATE EXTENSION IF NOT EXISTS postgis; through Alembic.
- Keep a direct or migration connection available for Alembic if the pooler rejects DDL or transaction settings.

Apply the schema

```
DATABASE_URL='postgresql+psycpg://...' .venv/bin/alembic upgrade head  
DATABASE_URL='postgresql+psycpg://...' .venv/bin/python -m api.scripts.seed_demo --yes-reset
```

Production warning

The seed command deletes marketplace rows. Run it only once on a new demo database. Disable the public seed route with ENABLE_DEMO_SEED=0.

8. Deploy the FastAPI backend to Vercel

- Import the Git repository as a new Vercel project.
- Set Root Directory to api.
- Leave framework detection and build command at their defaults. api/index.py exports the FastAPI app.
- Add environment variables for Production and Preview.

Variable	Production value	Purpose
DATABASE_URL	Pooled managed Postgres URL	Application database
ENABLE_DEMO_SEED	0	Disable destructive public seed
CORS_ORIGINS	https://YOUR-FRONTEND.vercel.app	Exact browser origin
CORS_ORIGIN_REGEX	Optional preview-domain regex	Preview deployments
DB_POOL_SIZE	2	Small per-instance pool
DB_MAX_OVERFLOW	1	Bound connection growth

- Deploy and open /health. It must report database connected.
- Open /docs and test product listing with no browser CORS dependency.
- Inspect Vercel Function logs for connection, import, or bundle errors.

9. Deploy the Next.js frontend to Vercel

- Import the same Git repository as a second Vercel project.
- Set Root Directory to apps/web.
- Confirm Framework Preset is Next.js.
- Set NEXT_PUBLIC_API_URL to the production backend URL without a trailing slash.
- Deploy, then copy the final frontend URL into the backend CORS_ORIGINS variable.
- Redeploy the backend because Vercel environment-variable changes do not affect older deployments.

End-to-end smoke test

Step	Expected result
Open frontend	Products load without API connection error
Open buyer demo	Buyer session and request ID appear
Find supply	Two tomato allocations total 100 kg
Create grouped order	One group ID and two farmer orders
Open farmer demo	Assigned order appears and can be accepted
Open admin demo	Users, listings, and orders are visible

10. Vercel monorepo release flow

Vercel supports connecting multiple projects to one repository. Each project uses its own Root Directory and URL.

Project	Root	Build/runtime	Primary environment
harvest-hub-web	apps/web	Next.js	NEXT_PUBLIC_API_URL
harvest-hub-api	api	Python / FastAPI Function	DATABASE_URL, CORS, pool limits

Safe release order

- 1. Create and review a backward-compatible Alembic migration.
- 2. Apply the migration to preview database.
- 3. Deploy preview backend and frontend; run smoke tests.
- 4. Apply the migration to production.
- 5. Deploy production backend, then frontend.
- 6. Monitor errors, latency, and database connections.
- 7. Remove old columns only in a later release after all code stops using them.

11. Testing and acceptance gates

Gate	Required check
Schema	alembic upgrade head succeeds on an empty PostGIS database
Persistence	Rows remain after backend restart
Spatial	Radius excludes remote farms and distances are plausible
Concurrency	Two confirmations cannot oversell one lot
Atomicity	Failure rolls back request, orders, and all quantities
Authorization	Buyer, farmer, and admin boundaries return 403 when crossed
Idempotency	Repeated confirmation cannot create duplicate grouped orders
Frontend	Buyer, farmer, and admin journeys pass in production
Operations	Backups, logs, alerts, and rollback procedure are verified

Current automated coverage

- Marketplace order state transitions.
- Seed persistence across sessions.
- PostGIS radius matching and multi-farmer allocation.
- Grouped-order creation and duplicate prevention.

Next test priority	Add HTTP integration tests and a concurrent reservation test using two independent database sessions.
---------------------------	---

12. Security and operations

- Replace long-lived bearer tokens with expiring sessions or signed access/refresh tokens.
- Rate-limit authentication, demo, and order-confirmation endpoints.
- Store secrets only in Vercel environment variables or the database provider's integration.
- Set `ENABLE_DEMO_SEED=0` and remove/reset known demo credentials for a real marketplace.
- Restrict database roles: runtime should not own schema migrations.
- Use TLS database connections and provider connection pooling.
- Record immutable order/status events for disputes and reputation.
- Set backup retention and practice point-in-time recovery.
- Add error monitoring and alerts for failed transactions, connection saturation, and 5xx rates.
- Keep the API and database in nearby regions to reduce latency.

13. Implementation plan

Milestone	Deliverable	Exit condition
A. Database activation	Start PostGIS, migrate, seed, run tests	All four integration tests pass
B. Repository boundaries	Move SQL queries out of routes	Route -> service -> repository -> DB
C. Reservation model	AVAILABLE/RESERVED/SOLD/RELEASED	Cancellation safely releases stock
D. Idempotency and audit	Request keys and domain events	Retries cannot duplicate effects
E. Routing	OSRM/GraphHopper adapter	Road time shown separately from radius
F. Transport pooling	Shipments, stops, vehicle capacity	Compatible orders share a route
G. Fulfilment	Delivery proof and disputes	Verified completion updates reputation
H. Payments	Mock provider, then fiat/Solana adapter	Settlement reconciles with database
I. Analytics	Historical events and dashboards	Reliable dataset supports forecasting

Most important engineering rule

Protect the order and inventory lifecycle. Every reservation, cancellation, fulfilment, and payment transition must be authorized, idempotent, auditable, and atomic where one database transaction can cover it.

14. Rollback and incident checklist

- Stop new confirmations if inventory consistency is uncertain.
- Preserve logs, request IDs, group IDs, and external payment references.
- Compare order quantities with lot reservations inside one read-only report.
- Roll back application code only when the schema remains backward-compatible.
- Use a forward migration to repair schema mistakes; avoid destructive downgrades after new writes.
- Restore from backup only after identifying the exact recovery point and expected data loss window.
- Re-run health, migration version, PostGIS version, matching, and grouped-order checks before reopening traffic.

Definition of successful deployment

Ready

Frontend and backend production URLs are live; /health confirms the database; Alembic is at head; PostGIS responds; seed or real records persist; buyer, farmer, and admin flows pass; secrets are not in Git; monitoring and recovery are configured.

15. Official references

Vercel Python runtime and FastAPI

- <https://vercel.com/docs/functions/runtimes/python>
- <https://vercel.com/kb/guide/ship-a-fastapi-app-on-vercel>

Vercel monorepos and projects

- <https://vercel.com/docs/monorepos>
- <https://vercel.com/docs/projects>

Environment variables and database operations

- <https://vercel.com/docs/environment-variables>
- <https://vercel.com/docs/postgres>
- <https://vercel.com/kb/guide/connection-pooling-with-functions>

Managed Postgres and PostGIS

- <https://vercel.com/marketplace/neon>
- <https://neon.com/blog/ten-most-popular-postgres-extensions>

Repository sources

- FARMER_MARKETPLACE_BUILD_GUIDE.md
- api/migrations/versions/0001_marketplace.py
- api/app/shared/store.py
- api/app/matching/engine.py
- api/app/orders/service.py
- apps/web/app/page.tsx